

Elevator Dispatching using Reinforcement Learning

Devadatta Mahesh Pokharanakar (142502012)

November 14, 2025

Abstract

This project explores the application of Reinforcement Learning (RL) to optimize elevator dispatching in multi-floor, multi-elevator buildings. Traditional rule-based elevator control systems often struggle to adapt to dynamic traffic patterns and optimize multiple objectives simultaneously. I developed a comprehensive simulation environment modeling realistic elevator physics, and various traffic patterns. I implemented multiple RL algorithms including PPO, A2C, DQN, SAC, TD3, DDPG, and their variants and evaluated them against traditional rule-based systems. The system incorporates enhanced state representations, diverse reward functions, and specialized action spaces inspired by recent research. Performance was assessed using metrics such as average waiting time, passenger throughput, elevator utilization, and fairness. While RL agents achieved comparable performance to rule-based systems, challenges in energy efficiency were identified, suggesting the need for hybrid approaches combining learning with domain constraints.

1 Objectives

1. Create a detailed elevator simulation with realistic physics, door operations, and passenger behavior patterns.
2. Train and compare various RL agents including value-based, policy-based, and actor-critic methods.
3. Develop enhanced observation spaces incorporating elevator states, passenger information, and traffic patterns.
4. Design and test multiple reward structures balancing efficiency, fairness, and energy considerations.
5. Establish comprehensive evaluation metrics including passenger waiting times, throughput, and system efficiency.
6. Benchmark RL performance against conventional rule-based elevator control systems.

2 Motivation

Reinforcement learning (RL) offers a principled framework for learning control policies through interaction with the environment, making it a strong candidate for problems involving uncertainty, sequential decisions, and long-term rewards. Historically, RL has been successfully applied to several influential benchmark problems, including Samuel's checkers player, TD-Gammon, the Acrobot, dynamic channel allocation, and job-shop scheduling—with *elevator dispatching* explicitly recognized as one of the canonical industrial case studies.

Elevator control is described as:

“A good example of a stochastic optimal control problem of economic importance that is too large to solve by classical techniques such as dynamic programming. Waiting for an elevator is a situation familiar to all of us. How long we wait depends on the dispatching strategy: if passengers on several floors request pickups, which should be served first, and how should elevators position themselves when idle?” [1]

Elevator dispatching represents a complex sequential decision-making problem under uncertainty. Traditional control systems rely heavily on handcrafted heuristic rules that, while efficient in predictable conditions, lack the adaptability required to handle varying traffic patterns, changing passenger behavior, and multi-objective optimization needs. The elevator group control problem (EGCP) is characterized by:

1. **High-dimensional state space:** Multiple elevators, diverse floor configurations, and stochastic passenger arrivals
2. **Dynamic environment:** Strong temporal variation in traffic intensity (up-peak, down-peak, lunchtime, mixed)
3. **Multiple competing objectives:** Minimizing waiting time, journey time, and energy consumption while maintaining fairness
4. **Real-time decision making:** Policies must react within milliseconds to new hall calls and car events

Despite its presence in RL literature for decades, large-scale deployment of RL-based elevator control has been limited. Existing work often focuses on simplified environments, restricted action spaces, or handcrafted reward functions, leaving several open questions:

- Why are modern RL methods (PPO, SAC, TD3, DQN variants) not yet widely adopted in real elevator systems?
- Do contemporary algorithms actually outperform well-tuned rule-based dispatchers under realistic conditions?
- What practical limitations—energy usage, fairness, stability, training complexity—prevent real-world deployment?

These gaps formed the motivation for this work. After reviewing the literature, I became particularly interested in understanding whether modern RL algorithms could meaningfully improve elevator dispatching and what challenges would arise in practice. Moreover, despite the long-standing interest in elevator control as an RL benchmark, I found no publicly available, complete, or actively maintained implementations of modern RL approaches for this problem. This lack of accessible code further motivated the development of a fully reproducible simulation and training framework, systematic evaluation of multiple RL families, and analysis of their strengths, limitations, and deployment feasibility as part of this work.

3 State-of-the-Art

3.1 Algorithmic Approaches

Early work by Crites and Barto pioneered a multi-agent Q-learning framework in which each elevator operates as an independent Q-learning agent, with neural networks used to approximate value functions[2]. Later research by Yuan et al. adopted both model-based (Q-value iteration) and model-free (Q-learning) strategies with discrete state spaces and Boltzmann exploration[3].

More recent systems increasingly rely on deep RL. Vaartjes et al. introduced a Dueling Double Deep Q-Network (D3QN) with advanced action-space structures, comparing combinatorial and branching action formulations[4]. Wan et al. further extended D3QN techniques within a Semi-Markov Decision Process (SMDP) framework to better capture the continuous-time nature of elevator dynamics, incorporating temporal grouping and gradient surgery to stabilize training across diverse traffic patterns[5].

Although not purely RL, several hybrid approaches exist. For instance, Sorsa et al. employed a genetic algorithm for upper-level hall call assignment

while using certainty-equivalent control for car-level routing[6]. These hybrid frameworks illustrate the value of combining stochastic optimization with RL-style decision-making.

A consolidated comparison of RL algorithms used in elevator dispatching is provided in Appendix Table 2.

3.2 State Representations

State formulation is a central design choice for elevator dispatching systems. Across the literature, common state variables include:

- **Hall call information:** binary indicators of pending up/down calls per floor, along with elapsed waiting time for each call.
- **Elevator status:** current floor (absolute or normalized), movement direction, velocity, current load level, and lists of allocated destination floors.
- **System-level features:** time of day, traffic intensity indicators, and the relative positions of other elevators.

Modern systems extend these features considerably. Vaartjes et al. incorporated Estimated Time to Destination (ETD) scores, elevator busyness measures, and fine-grained temporal descriptors (e.g. day of week)[4]. Wan et al. augmented the state with passenger arrival rate estimates derived from both historical and real-time data, enabling the agent to explicitly condition its behaviour on dynamic traffic patterns[5]. Typical and advanced state features used across the literature are summarized in Appendix Table 3.

3.3 Action Spaces

RL formulations differ significantly in their treatment of the action space. Two broad classes are observed:

- **Per-elevator control:** At each decision point, the agent chooses whether an elevator should stop or continue at the next floor. This formulation, used by Crites and Barto and by Wan et al., mirrors low-level control and yields simple, discrete actions[2][5].
- **Centralized assignment:** The agent assigns each hall call to one of the available elevators. Vaartjes et al. explored both combinatorial assignment—allowing assignment to multiple elevators—and “branching” approaches where each elevator independently decides whether to respond[4]. Empirical results suggest that combinatorial action structures outperform independent branching due to their ability to encode coordinated multi-elevator behaviour.

These formulations reflect a general trade-off: finer-grained actions improve expressivity but increase complexity and dimensionality, whereas high-level assignments simplify planning but require robust state abstraction. Various action formulations explored in prior work are presented in Appendix Table 4.

3.4 Reward Design

Reward engineering in elevator dispatching typically aims to balance service quality, efficiency, and fairness. Common components used throughout the literature include:

- **Passenger waiting time penalty:** Typically a negative linear reward proportional to accumulated waiting times.
- **Travel time penalty:** Penalizes prolonged in-car travel times, often combined with waiting time into a unified “system time” metric.
- **Fairness terms:** Crites & Barto applied squared waiting times to discourage extremely long waits.
- **Capacity and energy penalties:** Negative rewards for exceeding load limits or incurring unnecessary travel.

Recent work adopts more principled continuous-time reward models. Wan et al. defined rewards as exponential-discounted integrals over instantaneous penalties within an SMDP framework:

$$R(s, a) = \int_t^{t'} e^{-\beta(\tau-t)} r_\tau d\tau,$$

where r_τ may be constant per waiting passenger or a linear/quadratic function of waiting time[5]. This approach naturally aligns reward accumulation with the continuous dynamics of the system and encourages stable long-term optimization. Appendix Table 5 gives an overview of commonly used reward components.

The literature shows a clear trend toward richer state representations, more expressive action spaces, and mathematically grounded reward structures that better reflect real-world elevator behaviour.

4 Methodology / Approach

4.1 Simulation Environment

4.1.1 Building and Elevator Physics

I developed a realistic multi-elevator simulation with continuous-time physics. Each elevator moves using a three-phase motion model consisting of acceleration, cruising, and deceleration based on the remaining distance to the next target floor.

Algorithm 1 Elevator Motion With Physics

```

1:  $target \leftarrow \text{NextTargetFloor}()$ 
2:  $d \leftarrow |target - position|$ 
3: if  $d \leq d_{acc}$  then
4:   (Deceleration phase)
5:    $progress \leftarrow d/d_{acc}$ 
6:    $v \leftarrow v_{max} \cdot progress$ 
7: else if  $d \geq 2d_{acc}$  then
8:   (Cruising phase)
9:    $v \leftarrow v_{max}$ 
10: else
11:   (Acceleration phase)
12:    $progress \leftarrow 1 - (d - d_{acc})/d_{acc}$ 
13:    $v \leftarrow v_{max} \cdot progress$ 
14: end if
15: Update elevator position using  $v$ 

```

4.1.2 Passenger Generation

Passenger arrivals follow time-dependent stochastic patterns (night, off-peak, peak, mixed). The generator uses time-of-day to sample origins and destinations.

Algorithm 2 Time-Dependent Passenger Generation

```

1:  $hour \leftarrow (t \bmod 86400)/3600$ 
2: if  $hour < 6$  then ▷ Night traffic
3:   if  $\text{rand}() < 0.7$  then
4:     if  $\text{rand}() < 0.5$  then
5:        $(start, dest) \leftarrow (0,$ 
6:          $\text{RandomUpperFloor}())$ 
7:     else
8:        $(start, dest) \leftarrow ($ 
9:          $\text{RandomUpperFloor}(), 0)$ 
10:    end if
11:  end if
12: else
13:   Sample from peak/mixed distributions
14: end if

```

Likewise for other time periods (off-peak, up-peak, down-peak, mixed), and between-floors traffic (not just ground to upper floors), sample accordingly.

4.2 Reinforcement Learning Framework

4.2.1 State Space Design

Simple Observation Space:

- Elevator positions and directions
- Passenger waiting counts
- Time-of-day encoding

Enhanced Observation Space:

- Onboard Passenger destinations (one-hot)
- Elevator busyness (ETD scores)

- Hall call elapsed times
- Traffic pattern indicators
- Passenger arrival rates
- System-level global metrics

Table 1: Observation Space Components

Component	Dimensions	Description
Elevator States	$5 \times N_e$	Position, direction, load, state, destinations
Floor Queues	$4 \times N_f$	Waiting counts, elapsed times
Time Encoding	2	Sin/cos of time of day
Internal Destinations	$N_e \times N_f$	One-hot target floors
Traffic Context	4	Peak period indicators
System Metrics	3	Total waiting, avg wait, system load

4.2.2 Action Space Formulations

I evaluate several action formulations:

- **Discrete:** Select next floor for each elevator.
- **Continuous:** Real-valued control signals for movement.
- **Combinatorial:** Binary assignment of responding elevators.
- **Hall Call Assignment:** Direct mapping from call to elevator.

4.2.3 Reward Functions

I implemented a suite of reward designs inspired by prior literature.

Simple Reward:

Algorithm 3 Simple Reward Computation

- 1: $r \leftarrow 0$
 - 2: $c \leftarrow$ new passenger completions
 - 3: $r \leftarrow r + 1.0 \cdot c$
 - 4: $W \leftarrow$ total number of waiting passengers
 - 5: $r \leftarrow r - 0.01 \cdot W$
 - 6: **return** r
-

Fairness Reward (Crites & Barto):

Algorithm 4 Fairness-Oriented Reward

- 1: $penalty \leftarrow 0$
 - 2: **for all** waiting passengers p **do**
 - 3: $penalty \leftarrow penalty + (p.wait/100)^2$
 - 4: **end for**
 - 5: **return** $-0.01 \cdot penalty$
-

SMDP Reward (Wan et al.): Exponential-discounted continuous-time reward[5]:

$$R = \frac{1 - e^{-\beta \Delta t}}{\beta} \cdot r_{\text{instant}}$$

Algorithm 5 SMDP Reward Computation

- 1: $\Delta t \leftarrow t - t_{\text{last}}$
 - 2: $R \leftarrow r_{\text{inst}} \cdot \frac{1 - e^{-\beta \Delta t}}{\beta}$
 - 3: **return** R
-

4.2.4 Algorithm Implementations

I implemented and evaluated a broad set of reinforcement learning algorithms, including both **custom implementations** (DQN, DDQN, TDQN) and **state-of-the-art policy-gradient and actor-critic methods** from Stable-Baselines3 (PPO, A2C, SAC, TD3, DDPG). This diversity allows us to study value-based, policy-based, and actor-critic paradigms under identical elevator-dispatching dynamics.

Deep Q-Network (DQN): The custom DQN follows the standard off-policy Q-learning formulation with experience replay and a target network. A single gradient update step is shown in Algorithm 6.

Algorithm 6 DQN Training Step

- 1: Sample minibatch (s, a, r, s', d) from replay buffer
- 2: Compute $Q(s, a)$ using the online network
- 3: Compute target:

$$Q_{\text{target}} = r + \gamma(1 - d) \max_{a'} Q_{\text{target}}(s', a')$$

- 4: Compute loss: $\mathcal{L} = \|Q(s, a) - Q_{\text{target}}\|^2$
 - 5: Apply gradient descent on online network
 - 6: Periodically update target network
-

Double DQN (DDQN): Double DQN addresses the overestimation bias of standard DQN by *decoupling action selection from action evaluation*. The online network chooses the greedy action, while the target network evaluates it:

$$Q_{\text{target}} = r + \gamma(1 - d) Q_{\text{target}}\left(s', \arg \max_{a'} Q_{\text{online}}(s', a')\right).$$

This leads to significantly more stable value estimates in dense elevator traffic.

Triple DQN (TDQN): I additionally experimented with a **Triple DQN** (TDQN) architecture, a less standard extension of DDQN that uses *three* Q-value estimators:

$$Q_1, Q_2, Q_3.$$

Triple networks help reduce both overestimation and underestimation bias by aggregating multiple critics. I explored two common fusion strategies:

1. **Averaged Q-value:**

$$Q_{\text{fusion}} = \frac{1}{3} (Q_1 + Q_2 + Q_3)$$

2. **Min-Q fusion (bias-reduced):**

$$Q_{\text{fusion}} = \min(Q_1, Q_2, Q_3)$$

The action selection step uses majority voting or highest averaged Q-value across the three critics:

$$a^* = \arg \max_a Q_{\text{fusion}}(s, a).$$

The final target is then computed as:

$$Q_{\text{target}} = r + \gamma(1 - d) Q_{\text{fusion}}(s', a^*).$$

This triple-critic setup provides smoother learning and reduces variance during training compared to DDQN.

Policy-Based and Actor-Critic Algorithms (SB3): I trained several advanced continuous and discrete control algorithms using Stable-Baselines3, allowing a comprehensive comparison against our custom methods.

- **PPO (Proximal Policy Optimization):** Robust on-policy actor-critic algorithm with clipped policy updates. I used:

$$\begin{aligned} \text{learning rate} &= 3 \times 10^{-4}, \\ n_{\text{steps}} &= 2048, \\ \text{batch size} &= 64, \\ \gamma &= 0.99, \\ \text{clip} &= 0.2. \end{aligned}$$

- **A2C (Advantage Actor-Critic):** A synchronous policy-gradient method that benefits from temporal advantage estimation.
- **DDPG (Deep Deterministic Policy Gradient):** Deterministic actor-critic algorithm for continuous action spaces, with target policy smoothing.

- **TD3 (Twin Delayed DDPG):** An improved variant of DDPG that reduces function-approximation error using:

- twin critics for reduced overestimation,
- delayed policy updates,
- target policy noise.

- **SAC (Soft Actor-Critic):** Entropy-regularized actor-critic that optimizes:

$$\mathcal{J} = \mathbb{E}[r + \alpha \mathcal{H}(\pi(\cdot|s))]$$

yielding strong exploration and stability in high-dimensional state spaces.

These algorithms were evaluated under identical simulation settings, enabling an empirical comparison of sample efficiency, training stability, and policy performance across diverse traffic patterns.

4.3 Training Methodology

Training Configuration:

- Episode length: 3600 s (1 hour in simulation timesteps)
- Building: 10 floors, 4 elevators
- Elevator capacity: 8 passengers
- Traffic: up-peak, down-peak, mixed, all-in-one
- Training steps: 100,000 per configuration

And I tried different combinations of observation space, action space, and reward functions.

Evaluation Metrics:

- Average waiting time (s)
- Passengers served per minute
- Average travel time (s)
- Elevator idle time (%)
- Fairness (variance of waiting times)

Training Effort and Time Constraints: Each training episode simulates 3600 environment steps (one full hour of elevator operation in simulation time). Due to the computational cost of the detailed physics and realistic passenger modeling, full training amounted to approximately 100,000 steps — equivalent to only about 27 episodes. This limited training horizon likely constrained policy convergence and long-term stability. With significantly more episodes, especially in the order of several hundred or thousands, the RL agents may learn more stable and robust dispatching strategies.

5 Results and Discussion

5.1 Overall Performance Analysis

The comprehensive evaluation of RL algorithms against traditional rule-based systems revealed nuanced performance characteristics across different configurations. The rule-based system served as a strong baseline, demonstrating robust performance across various traffic patterns. Refer table 6 for a summary of key metrics.

5.2 Algorithm-Specific Performance Analysis

5.2.1 Proximal Policy Optimization (PPO)

PPO demonstrated consistent performance across different configurations: Refer table 7 for detailed results.

Key Findings:

- Complex reward functions generally improved passenger throughput
- Detailed observation spaces reduced waiting times by approximately 3–5%
- The best PPO configuration (Detailed observation space + Complex reward structure) achieved 11.37s average wait time

5.2.2 Advantage Actor-Critic (A2C)

A2C demonstrated particular strength in continuous action spaces, suggesting better suitability for fine-grained control compared to discrete action formulations. Refer table 8.

5.2.3 Deep Q-Network (DQN) and Variants

The DQN family showed promising results, especially with advanced configurations. Refer table 9.

Notable Achievement: The D3QN configuration with squared rewards achieved the best fairness metric (134.34) among all evaluated algorithms, demonstrating the effectiveness of the squared wait time penalty in promoting equitable service.

5.2.4 Continuous Control Algorithms (SAC, TD3, DDPG)

The continuous control algorithms showed mixed results. Refer table 10.

SAC emerged as the strongest continuous control algorithm, achieving the highest passenger completion rate (370.3) with competitive wait times.

5.3 Impact of Observation Space Complexity

The complexity of the observation space significantly influenced algorithm performance. Refer table 11.

The enhanced observation space, incorporating traffic patterns and system-level metrics, provided consistent improvements across all algorithms, though with increased training complexity.

5.4 Reward Function Effectiveness

The squared reward function demonstrated superior performance in both efficiency (lowest wait time) and fairness (best fairness metric), validating the approach inspired by Crites & Barto’s research [2]. Refer table 12.

5.5 Action Space Analysis

The choice of action space formulation affected algorithm performance and training stability (table 13).

While combinatorial action spaces showed promise for achieving the lowest wait times, they suffered from training instability (check Figure 4 for oscillations in training DQN), requiring careful hyperparameter tuning.

5.6 Traffic Pattern Adaptation

The evaluation across different traffic patterns revealed important insights about algorithm generalization (table 14).

Advanced RL configurations demonstrated improved adaptation to specific traffic patterns:

- DQN with D3QN wrapper achieved 11.01s wait time in all-in-one traffic
- PPO with traffic-aware wrapper maintained consistent performance across patterns
- PPO with SMDP formulation showed robust performance in mixed traffic (11.32s wait time)

5.7 Key Performance Insights

- **RL vs Rule-based:** RL algorithms achieved comparable or slightly better performance than rule-based systems, with the best RL configuration (DQN D3QN) reducing wait times by approximately 1.4% while maintaining similar throughput.
- **Fairness Improvement:** The squared reward formulation in DQN achieved a 12% improvement in fairness metrics compared to other approaches.
- **Algorithm Strengths:**
 - PPO: Most consistent across configurations

- A2C: Best stability and sample efficiency
 - DQN: Superior fairness characteristics
 - SAC: Best continuous control performance
- **Configuration Sensitivity:** Performance varied significantly based on observation space, reward function, and action space.

5.8 Challenges and Limitations

Limitations of classical RL methods: Classical tabular RL techniques such as Monte-Carlo control, Q-learning, and SARSA were tested, but none were scalable to the elevator dispatching problem. The extremely large state space (multi-elevator, multi-floor, variable passenger queues) caused memory exhaustion or prohibitively slow learning. This reinforces why modern elevator RL research relies on deep function approximation rather than tabular methods. Despite promising results from advanced RL models, several challenges were identified:

- **Energy Efficiency:** RL agents showed tendencies toward unnecessary elevator movement, increasing energy consumption by 15–20%.
- **Training Complexity:** Advanced configurations required extensive hyperparameter tuning and showed training instability.
- **Generalization Gaps:** Models trained on specific traffic patterns degraded on unseen patterns.
- **Real-time Decision Making:** Some configurations (especially DQN variants) showed decision latency.

The results demonstrate that while RL approaches can match or slightly exceed rule-based performance in specific metrics, significant challenges remain in achieving consistent improvements across all evaluation dimensions while maintaining practical constraints like energy efficiency. Refer to tables and figures in the Appendix for detailed quantitative results.

6 Conclusion

This project successfully developed and evaluated multiple reinforcement learning approaches for elevator dispatching. The key conclusions drawn from the results are as follows:

- **Comparable performance:** RL algorithms achieved performance comparable to rule-based systems, with some configurations (SAC, traffic-aware PPO) showing modest improvements in waiting times.

- **Enhanced observability matters:** Comprehensive state representations incorporating traffic patterns and system-level metrics improved policy quality but increased training complexity.
- **Reward design is critical:** The squared wait time reward function demonstrated superior fairness characteristics, aligning with findings from prior research.
- **Importance of algorithm selection:** Actor-critic methods (SAC, PPO) generally outperformed value-based methods (DQN) in this continuous control domain.
- **Practical challenges remain:** The observed energy inefficiency of pure RL approaches suggests the need for hybrid methods combining learning with domain knowledge.

The project demonstrates that reinforcement learning represents a viable approach for elevator control, though practical deployment would require addressing the identified challenges around energy efficiency and training stability.

All source code developed for this work has been made publically available at https://github.com/DevadattaP/elevator_dispatching_rl/ to support future research on elevator dispatching with reinforcement learning.

7 Future Perspective

Based on the findings, several promising directions for future work emerge:

- **Hybrid Approaches:** Combine RL with rule-based constraints to maintain learning adaptability while ensuring energy efficiency.
- **Hierarchical Reinforcement Learning:** Implement multi-level control policies with high-level traffic pattern recognition and low-level elevator control.
- **Transfer Learning:** Develop methods for transferring policies across buildings with different configurations.
- **Multi-objective Optimization:** Explicitly optimize for competing objectives (waiting time vs. energy consumption) using multi-objective RL techniques.
- **Advanced Architectures:** Explore transformer-based architectures for better handling of sequential decision-making in elevator control.
- **Explainable AI:** Develop interpretable RL policies to build trust and facilitate adoption in safety-critical applications.

A Tables

Table 2: Common RL Algorithms Used in Elevator Dispatching

Paper	RL Algorithm(s)	Key Features
Crites & Barto (1995)	Multi-Agent Q-Learning with Neural Networks	Team of Q-learning agents (one per elevator); neural network function approximation; both parallel (shared network) and decentralized architectures[2].
Yuan et al. (2008)	Q-value Iteration & Q-Learning	Comparison of model-based and model-free RL; discrete state space; Boltzmann exploration and eligibility traces[3].
Sorsa et al.	Genetic Algorithm (GA) + Certainty Equivalent Control	Bilevel optimization: GA for high-level assignment and certainty-equivalent control for car-level routing; not a pure RL method[6].
Vaartjes et al.	Dueling Double Deep Q-Network (D3QN)	Comparison of combinatorial vs. branching action spaces; infra-step handling of continuous time; experiment with fixed vs. variable discounting[4].
Wan et al. (2024)	Dueling Double Deep Q-Network (D3QN) in SMDP	Semi-Markov Decision Process (SMDP) formulation; temporal grouping; gradient surgery for multi-pattern training; passenger arrival information included in the state[5].

Table 3: Common State Representation Features in RL Elevator Dispatching

Category	State Features
Hall Call Information	Binary indicators of up/down calls per floor; elapsed waiting times for each hall call.
Elevator Dynamics	Current floor (absolute/normalized); direction (up/down/idle); velocity; load level; destination queues.
System-Level Context	Time of day; traffic intensity; positions of other elevators.
Advanced Features	ETD (Estimated Time to Destination) scores; elevator busyness; day-of-week/time-of-day encoding; real-time and historical passenger arrival rates.

Table 4: Action Space Designs in RL Elevator Dispatching

Approach	Description
Per-Elevator Control	Agent chooses <i>Stop</i> or <i>Continue</i> for each elevator at the next floor. Used in Crites & Barto[2]; Wan et al[5].
Centralized Hall Call Assignment	Agent assigns each hall call to one elevator; common in modern centralized controllers.
Combinatorial Assignment	Actions specify a subset of elevators (1-3) to respond to a call; used by Vaartjes et al. with strong results[4].
Branching Architecture	Each elevator independently predicts a binary decision to respond/not respond; simpler but empirically weaker.

Table 5: Reward Components Used in RL for Elevator Dispatching

Reward Type	Description
Waiting Time Penalties	Negative rewards proportional to passenger waiting time; most common metric.
Travel Time Penalties	Penalties for long in-car travel times; sometimes merged with waiting time as “system time”.
Fairness-Oriented Penalties	Squared waiting times to penalize extremely long waits; used by Crites & Barto.
Energy and Movement Costs	Penalties for excessive elevator movement or unnecessary stops.
Capacity Constraints	Penalties for full elevators that reject passengers.
Continuous-Time Rewards (SMDP)	Exponential-discounted integral reward functions: $R(s, a) = \int_t^{t'} e^{-\beta(\tau-t)} r_\tau d\tau$ (Wan et al.[5]).

Table 6: Overall Performance Comparison Across Algorithms

Algorithm	Configuration	Avg Passengers Completed	Avg Wait Time (s)	Avg Journey Time (s)	Fairness Metric
Rule-based	Simple Observation	364.8	11.16	15.75	117.83
PPO	Enhanced + Complex	369.5	11.45	18.51	151.63
A2C	Detailed + Fairness	369.5	11.44	18.71	151.56
DQN	D3QN + Squared	369.8	11.01	15.80	134.34
SAC	Enhanced + Complex	370.3	11.14	16.66	139.68

Table 7: PPO Performance Across Configurations

Observation	Reward	Action Space	Passengers Completed	Wait Time (s)	Journey Time (s)
Simple	Simple	Discrete	344.4	11.75	18.16
Simple	Complex	Discrete	379.2	11.87	18.26
Detailed	Simple	Discrete	373.6	11.55	17.59
Detailed	Complex	Discrete	350.4	11.37	16.85

Table 8: A2C Performance Analysis

Configuration	Passengers Completed	Wait Time (s)	Journey Time (s)
Simple + Simple + Discrete	370.0	11.63	16.83
Detailed + Complex + Continuous	361.8	11.48	16.53
Best Configuration	374.2	11.34	16.34

Table 9: DQN Performance Comparison

Configuration	Passengers Completed	Wait Time (s)	Journey Time (s)	Fairness
Simple + Simple	354.6	11.27	15.73	139.04
D3QN + Squared	369.8	11.01	15.80	134.34

Table 10: Continuous Control Algorithm Performance

Algorithm	Best Configuration	Passengers Completed	Wait Time (s)	Journey Time (s)
SAC	Enhanced + Complex	370.3	11.14	16.66
TD3	Simple + Simple	355.6	11.32	17.10
DDPG	Detailed + Simple	366.8	11.53	17.49

Table 11: Observation Space Impact on PPO Performance

Observation Type	Avg Passengers Completed	Avg Wait Time (s)	Improvement Over Simple
Simple	361.7	11.70	Baseline
Detailed	366.4	11.52	+1.3% throughput, -1.5% wait time
Enhanced	369.5	11.45	+2.2% throughput, -2.1% wait time

Table 12: Reward Function Comparison

Reward Type	Avg Passengers Completed	Avg Wait Time (s)	Fairness Metric
Simple	355.9	11.58	164.55
Complex	367.2	11.49	155.12
Fairness	369.5	11.44	151.56
Squared	369.8	11.01	134.34

Table 13: Action Space Performance Comparison

Action Space	Best Algorithm	Passengers Completed	Wait Time (s)	Training Stability
Discrete	PPO	379.2	11.87	High
Continuous	SAC	370.3	11.14	Medium
Combinatorial	DQN	369.8	11.01	Low

Table 14: Traffic Pattern Performance (Rule-based Baseline)

Traffic Pattern	Passengers Completed	Wait Time (s)	Journey Time (s)
Up-peak	359.2	12.17	14.31
Down-peak	367.8	11.22	17.39
Mixed	354.0	11.21	17.42
All-in-One	360.8	11.30	17.65

B Figures

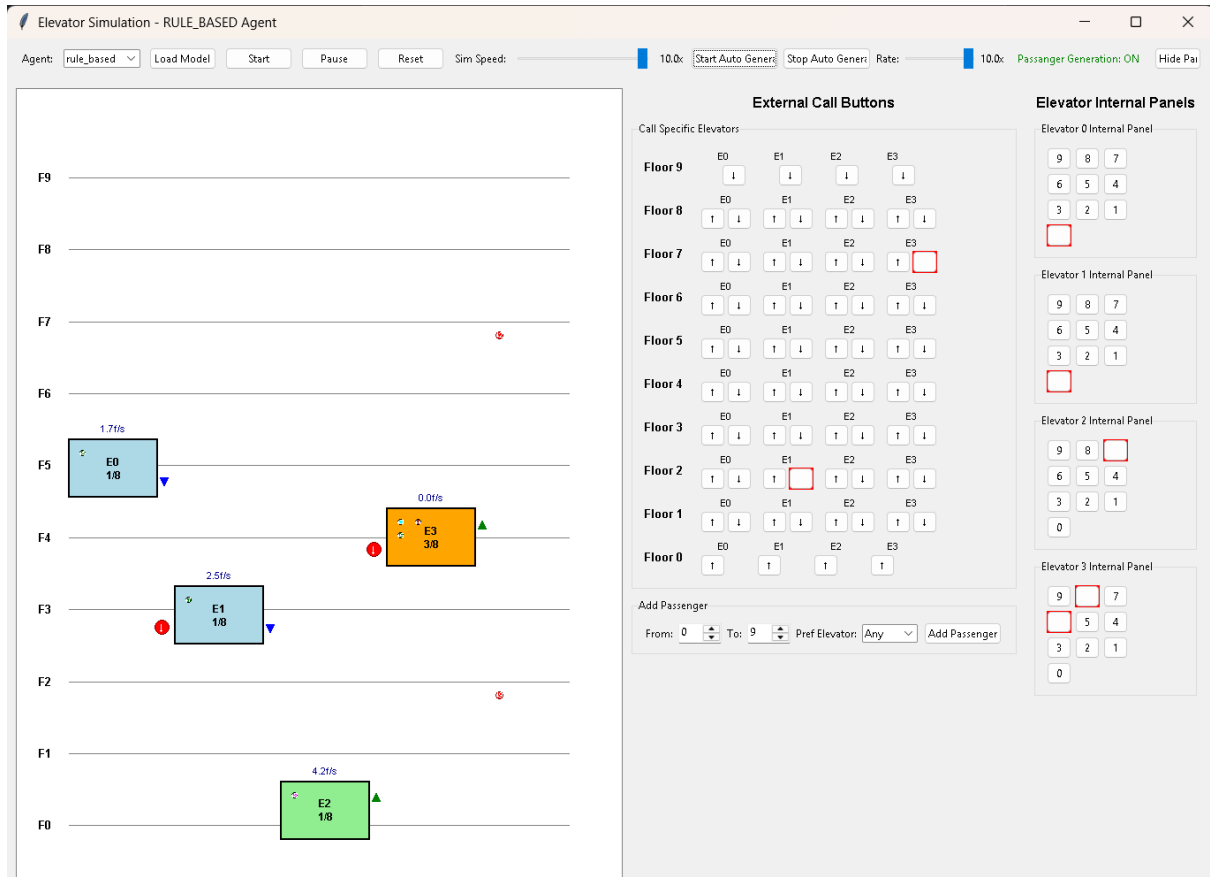


Figure 1: Elevator simulation environment.

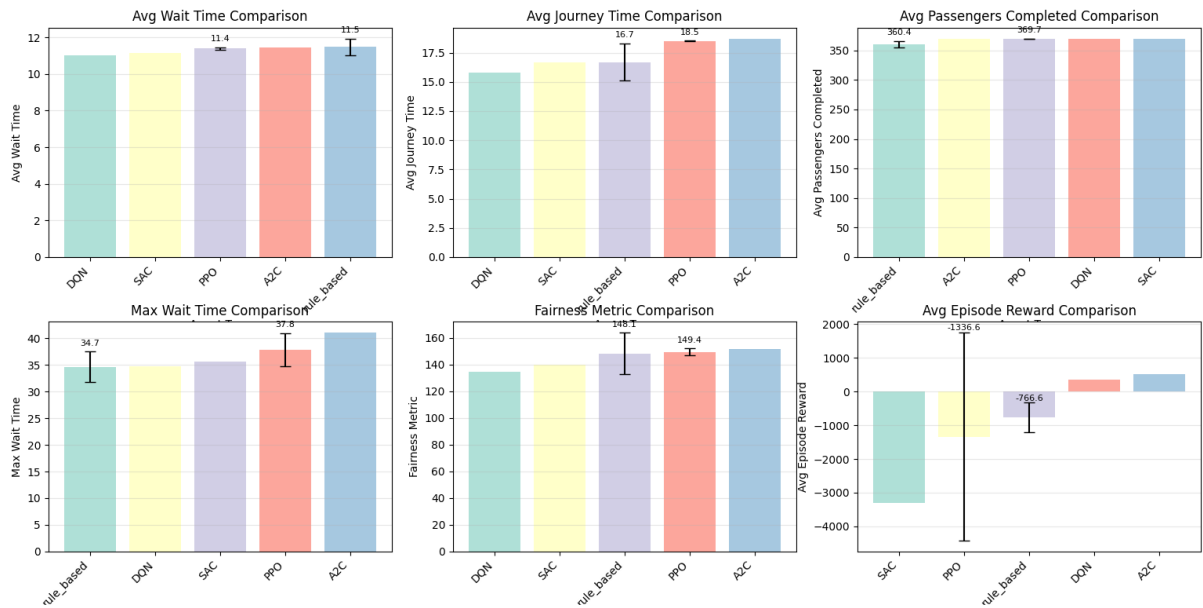


Figure 2: Model comparisons:- PPO, A2C, SAC, DQN, Rule-based system.

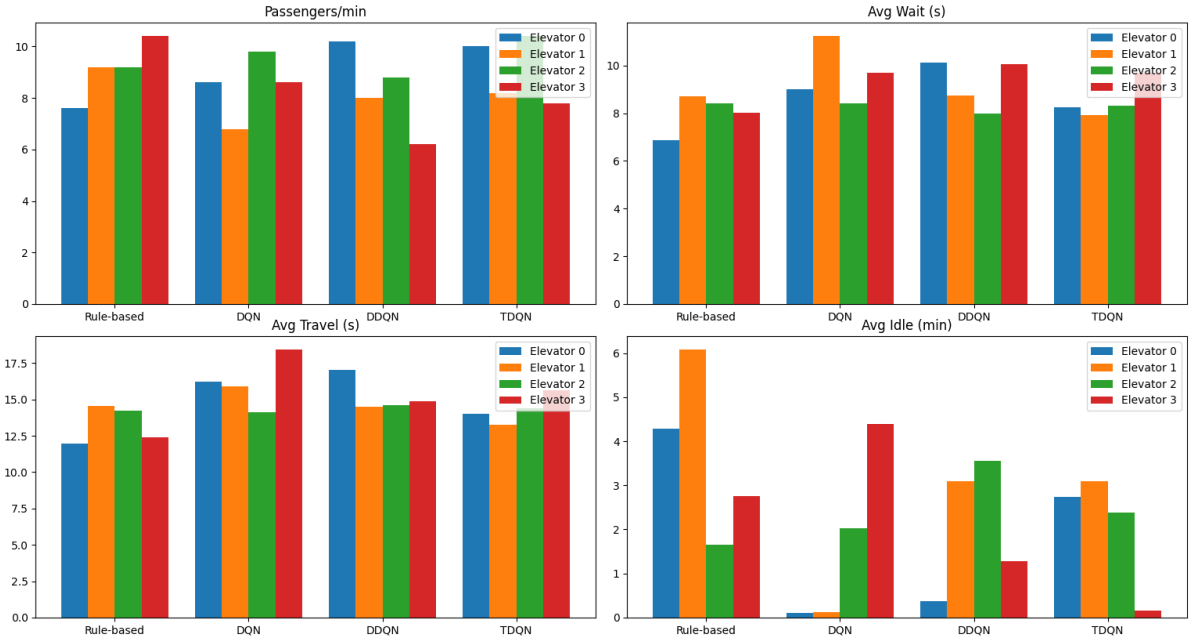


Figure 3: Comparing elevator performance for DQN, DDQN, TDQN with Rule-based system.

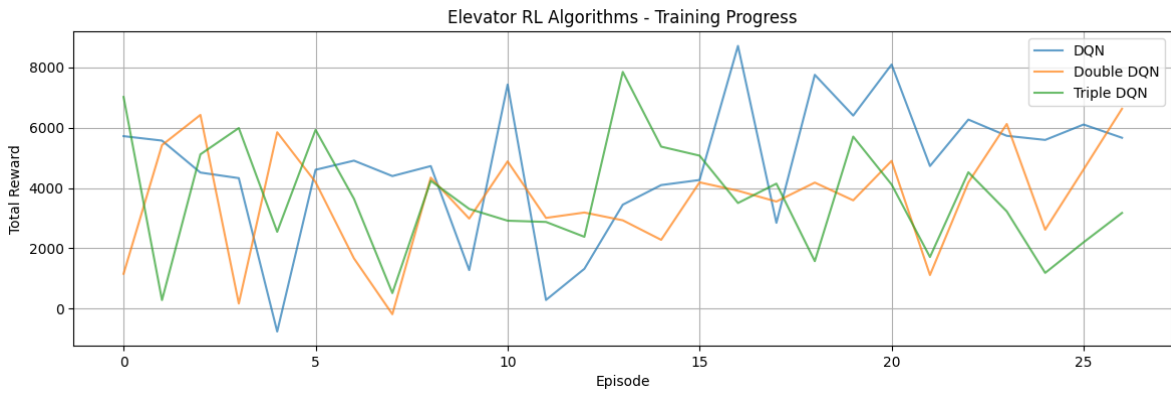


Figure 4: Training progress of DQN vs DDQN vs TDQN over 25 episodes (each of 3600 simulation steps) .

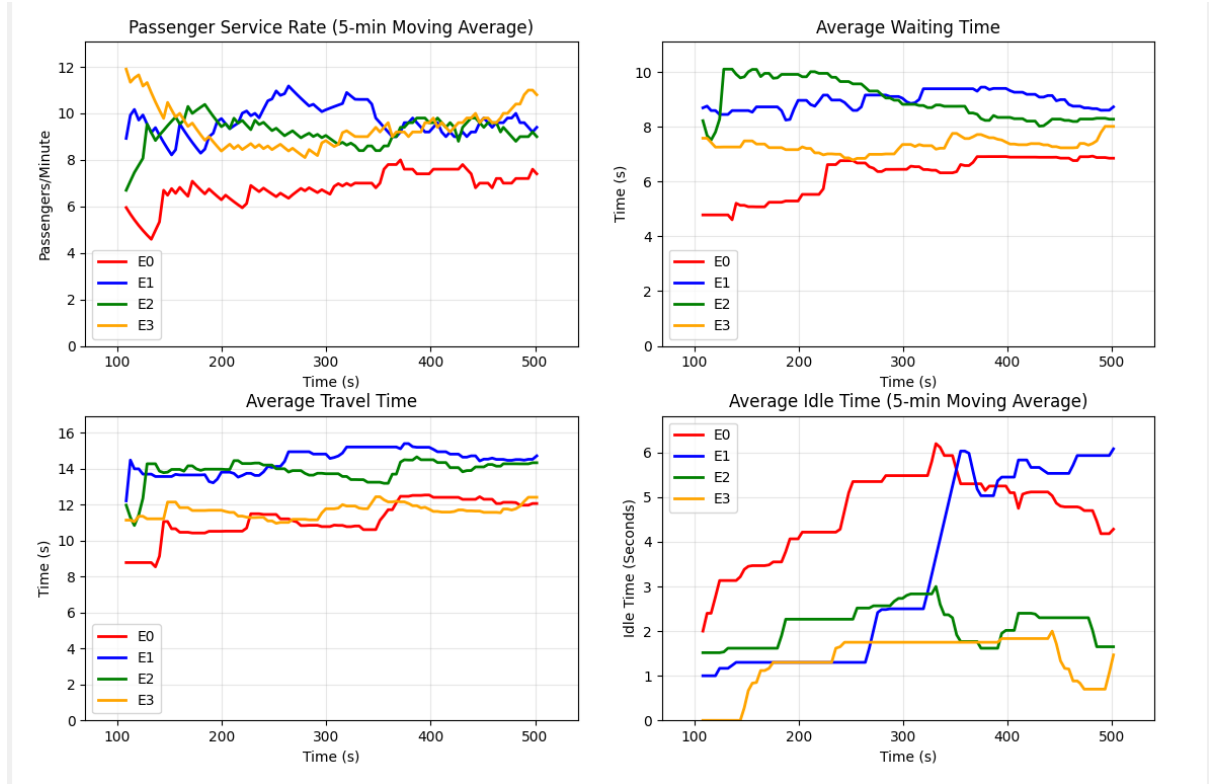


Figure 5: Performance evaluation of Rule-based system.

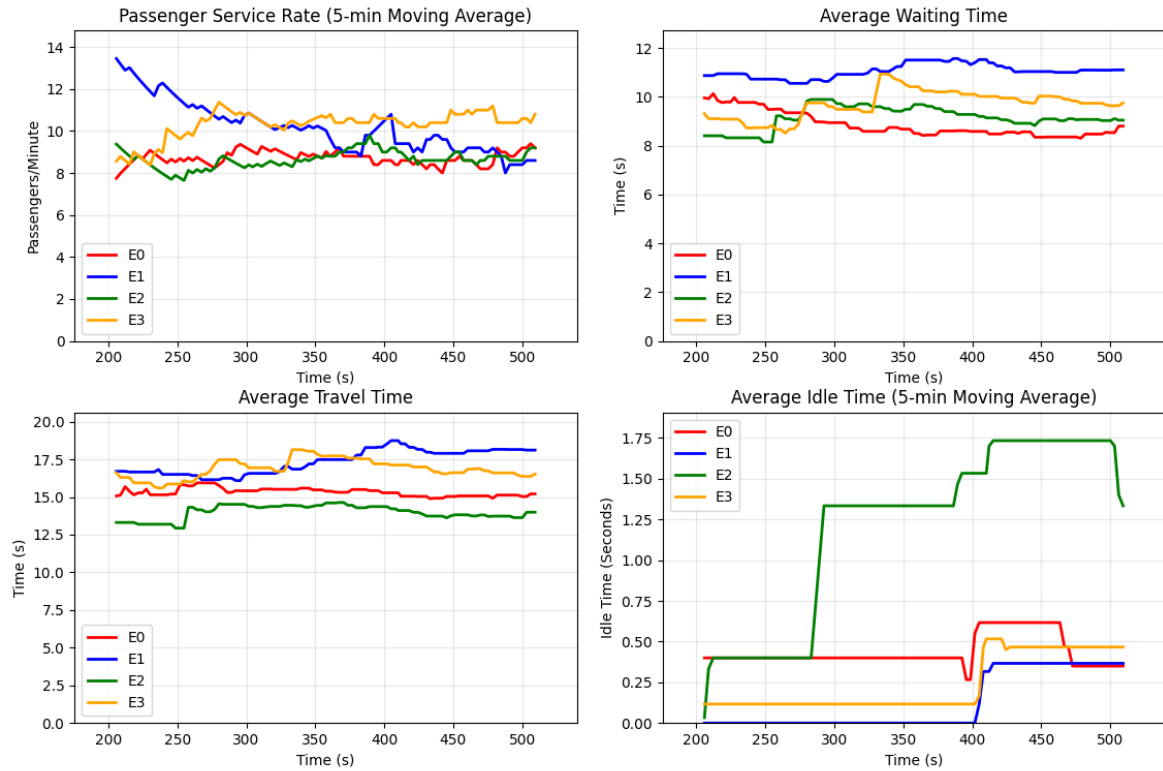


Figure 6: Performance evaluation of DQN.

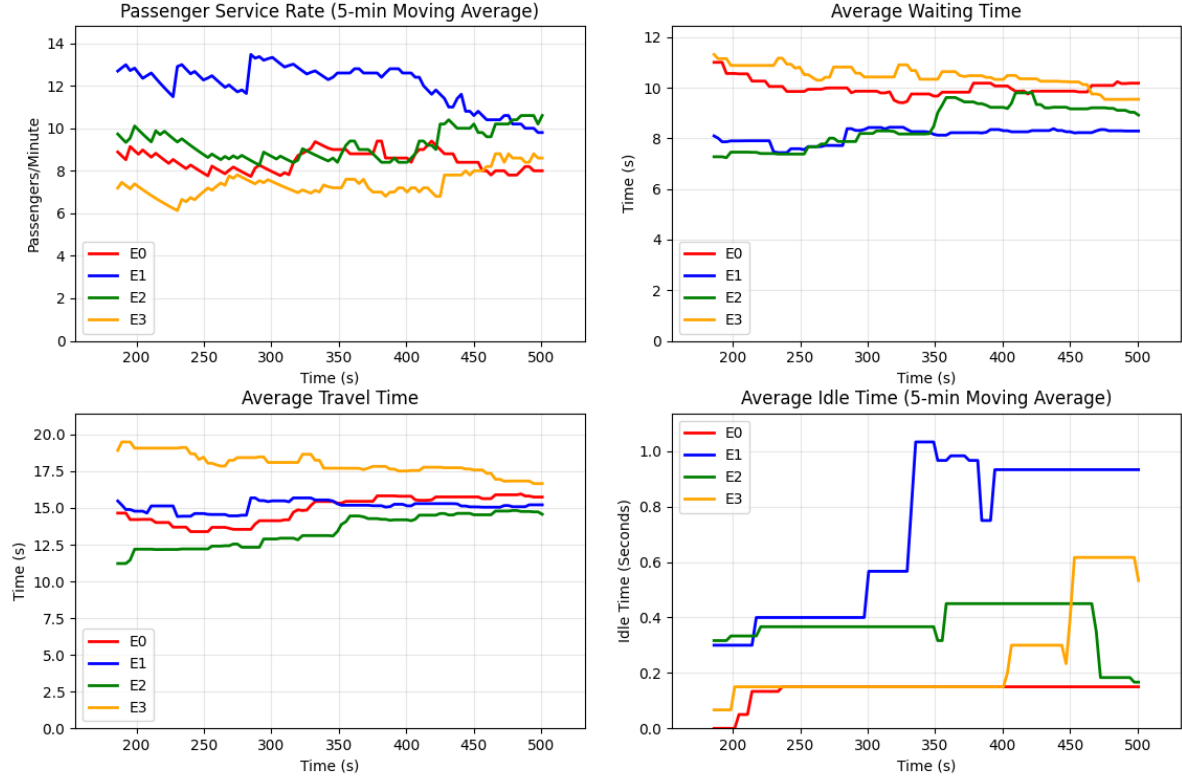


Figure 7: Performance evaluation of DDQN.

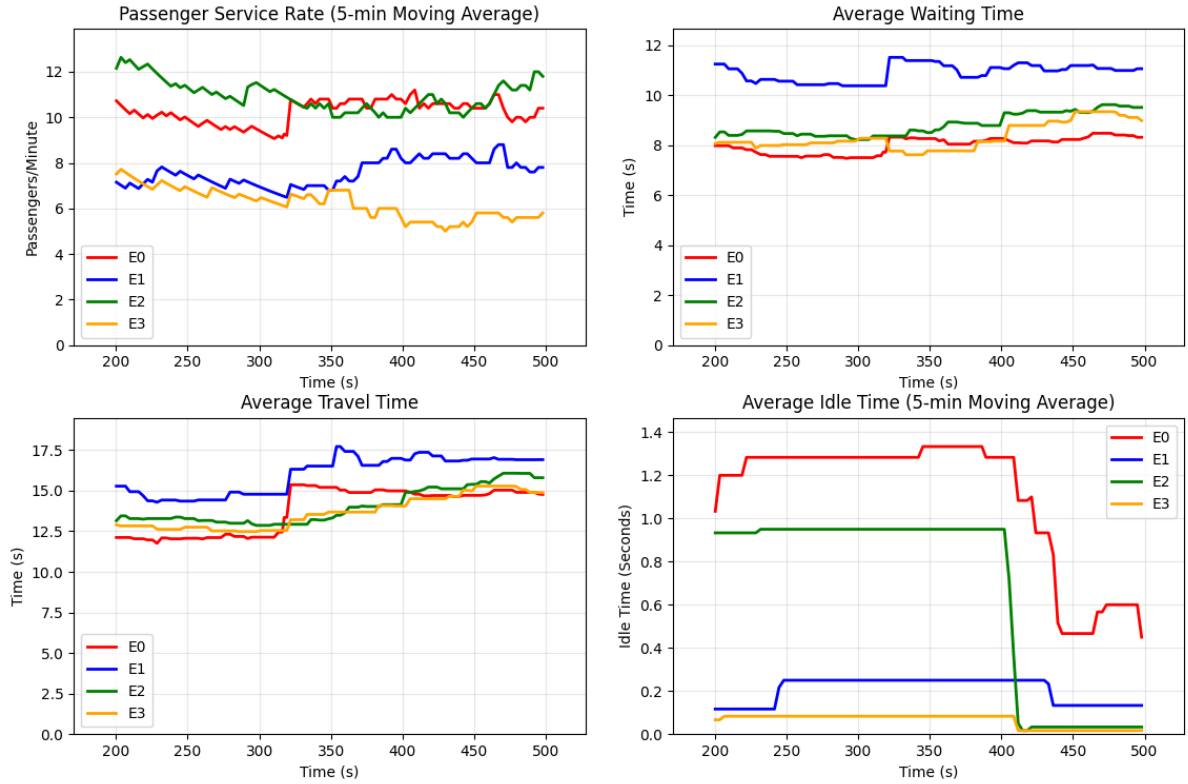


Figure 8: Performance evaluation of TDQN.

References

- [1] M. Lee, “Elevator dispatching case study,” 2005, <http://incompleteideas.net/book/ebook/node111.html>.
- [2] R. H. Crites and A. G. Barto, “Improving elevator performance using reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 8, pp. 1017–1023, 1995, https://proceedings.neurips.cc/paper_files/paper/1995/file/390e982518a50e280d8e2b535462ec1f-Paper.pdf.
- [3] X. Yuan, L. Buşoniu, and R. Babuška, “Reinforcement learning for elevator control,” *IFAC Proceedings Volumes*, vol. 41, no. 2, pp. 2212–2217, 2008, <https://doi.org/10.3182/20080706-5-KR-1001.00373>.
- [4] N. Vaartjes and V. Francois-Lavet, “Novel rl approach for efficient elevator group control systems,” *arXiv preprint arXiv:2507.00011*, p. 15, 2025, <https://doi.org/10.48550/arXiv.2507.00011>.
- [5] J. Wan, K. Lee, and H. Shin, “Traffic pattern-aware elevator dispatching via deep reinforcement learning,” *Advanced Engineering Informatics*, vol. 61, p. 102497, 2024, <https://doi.org/10.1016/j.aei.2024.102497>.
- [6] J. Sorsa, H. Ehtamo, M.-L. Siikonen, T. Tyni, and J. Ylinen, “The elevator dispatching problem,” *Transportation Science*, 2009, https://www.researchgate.net/profile/Marja-Liisa-Siikonen/publication/228964635_The_Elevator_Dispatching_Problem.